Tool Use III: Reliability, Idempotency, and Receipts

FRIDAY AI Education — Episode 036 · 2026-07-22 · Printable Companion

FRIDAY AI Education — Episode 036 · Tool Use, Part 3 of 3 · Lesson 22 of 37

Abstract

The previous two lessons established that a language model becomes useful when it can act through tools, and that those tools must be governed by permissions and approval gates. This paper addresses the question those lessons deliberately postponed: what happens when an action *goes wrong in the middle*. Real systems fail partially — the network drops after the payment was charged but before the confirmation arrived; the scheduler fires twice; the model calls the same tool again because it never saw the first result. An agent that acts in the world without engineering for these cases is not automated; it is merely fast at making messes.

This paper teaches the five disciplines that turn a tool call into a reliable operation: **validation** (check the request before acting), **idempotency** (design actions so repeating them is harmless), **retry** (repeat failed actions deliberately, not accidentally), **receipts** (record what actually happened, durably), and **verification** (confirm the world changed the way you intended). Together they form a loop that is the difference between a demo and a system you would let touch money, customers, or your reputation. We work through three concrete cases — a feed publisher, a scheduled job, and a database write — and close with the industrial-strength version of this idea, durable execution, as practiced by Temporal.

1. The Bridge: From Permission to Reliability

Tool Use II answered the question *"should this action be allowed?"* Least privilege, scoped permissions, approval gates — all of that governs whether an action may happen.

Today's question is different and comes after it in every real workflow: *given that an action is allowed, how do I know it actually happened — once, correctly, and provably?*

This distinction matters more for agents than for traditional software, for one specific reason. A human operator who clicks "publish" and gets a spinning wheel will pause, check the site, and decide whether to click again. A language model driving a tool has no such instinct unless you build it. When a tool call times out, the model sees nothing — and "nothing" is ambiguous. The action may have failed before starting, failed halfway, or fully succeeded with only the confirmation lost in transit. Each of these requires a different response, and the model cannot tell them apart from silence.

So the subject of this paper is not intelligence. It is plumbing. But it is the plumbing that determines whether you can ever hand an agent a task that matters.

2. Why Actions Fail: The Uncomfortable Physics of Distributed Systems

Every tool call an agent makes crosses at least one boundary: a network hop to an API, a write to a database, a message to a queue. Boundaries are where reliability goes to die, and they fail in three distinct shapes. It is worth internalizing these shapes because everything else in this paper is a response to one of them.

Failure before the action. The request never arrived. The API was down, the DNS lookup failed, the auth token expired. This is the *pleasant* failure mode: nothing happened, the world is unchanged, and you can simply try again. Most engineers' mental model of failure is this shape, which is precisely the problem.

Failure after the action. The request arrived, the action completed, and the *response* was lost. The payment went through; the confirmation didn't make it back. From the caller's perspective this is indistinguishable from the first case — a timeout is a timeout — but the correct response is the opposite. Retrying here means doing the thing twice.

Failure during the action. The action was partially applied. Three of five database rows were written. The file uploaded but the index entry pointing to it didn't. The email was sent to half the list. The system is now in a state that neither "before" nor "after" describes, and naive retries may make it worse rather than better.

Traditional software engineering has spent fifty years developing responses to these three shapes: transactions, exactly-once messaging illusions, reconciliation jobs. Agents inherit all of these problems and add one of their own: **the actor is a probabilistic text generator**. A model may call the wrong tool, call the right tool with wrong arguments, call it twice because its context got confused, or hallucinate that it already called it. The failure surface is the union of distributed-systems failures and model failures.

The good news is that the same five disciplines handle both. That is not a coincidence — they were designed for unreliable actors, and it turns out it doesn't matter much whether the unreliable actor is a flaky network or a flaky reasoner.

3. Validation: Check Before You Act

Validation is the cheapest discipline and the one most often skipped, because in the happy path it appears to do nothing.

The principle: **before executing an action, check that the request is well-formed, plausible, and consistent with the current state of the world.** Not after. Before. Every dollar spent on validation buys ten dollars of not-cleaning-up.

For agent-driven tools, validation happens in layers:

Schema validation. Does the request have the right shape? The model asked to publish a post — is there a title, a body, a target feed that exists? This catches the model's most common failure: syntactically confident, semantically wrong arguments. A tool that accepts loosely-typed input from a language model without schema validation is an open invitation for garbage to reach production. Anthropic, OpenAI, and every serious tool-use framework enforce JSON schemas at the tool boundary for exactly this reason.

Semantic validation. Is the request *plausible*? A payout of €4.50 and a payout of €450,000 both validate against a schema that says "amount: number." Plausibility checks — bounds, rate ceilings, "this is 40× the usual value, stop" — are where you encode business judgment into the tool layer. Stripe's fraud tooling, at its core, is semantic

validation at planetary scale: the transaction is well-formed, but does it *make sense*?

Precondition validation. Is the world in a state where this action is coherent? Publishing a post that's already published, refunding a payment that was already refunded, deleting a record that doesn't exist. Precondition checks read current state before writing new state — which, note, requires the tool to be able to *read* as well as write. A write-only tool cannot validate its own preconditions, which is a design smell you'll now start noticing everywhere.

One founder-relevant framing: validation is where you decide *how much you trust the caller*. In Tool Use II the caller was governed by permissions — what it *may* do. Validation governs what it may do *right now, with these arguments, given this state*. Permissions are static; validation is dynamic. You need both, and they are not substitutes.

4. Idempotency: Design Actions So Repeating Them Is Harmless

This is the core concept of the episode, and the one worth carrying around for the rest of your career.

An operation is **idempotent** if performing it twice has the same effect as performing it once. Setting a thermostat to 20° is idempotent — do it five times, the room is 20°. Turning the thermostat *up by one degree* is not — do it five times, you're sweating. `Set status to published` is idempotent. `Append to feed` is not.

Why does this matter so much? Return to the second failure shape from Section 2: the action succeeded but the confirmation was lost. The caller — human, scheduler, or agent — cannot know this happened. The only universally safe response to ambiguity is to try again. **But trying again is only safe if the operation is idempotent.** Idempotency is what converts "I don't know if it worked" from a crisis into a shrug: just do it again, and the worst case is a no-op.

This gives you the single most important design rule in this paper:

> **Retries are only safe on top of idempotent operations. If you cannot make an operation idempotent, you cannot safely retry it, and if you cannot safely retry it, you cannot safely automate it.**

Some operations are *naturally* idempotent: setting a value, deleting by ID (deleting an already-deleted record is a no-op), overwriting a file at a known path. Prefer these shapes when designing tools. A surprising fraction of non-idempotent operations are just idempotent operations phrased badly — "increment the counter" is often "set the counter to what I computed," and "append the post" is often "ensure a post with this ID exists."

For operations that are *not* naturally idempotent — charging a card, sending an email, creating a record with a generated ID — the standard industrial technique is the **idempotency key**. The caller attaches a unique token to the request: "create this charge, idempotency key `abc123`." The server records which keys it has already processed. If the same key arrives again — because of a network retry, a confused scheduler, or a model that lost the plot — the server recognizes it and returns the *original* result instead of executing again. The operation has been made idempotent by construction.

Stripe made this famous: every mutating call to their API accepts an `Idempotency-Key` header, precisely because the alternative — a payments API where a timeout might mean a double charge — is commercially unacceptable. The pattern is now table stakes in payments, messaging, and infrastructure APIs. When you evaluate a vendor's API and it has no idempotency mechanism on its mutating endpoints, you have learned something about how seriously they take your money.

For agent systems, idempotency keys have an elegant extra property: **the key can come from the workflow, not the model.** If the orchestration layer stamps each intended action with a deterministic key

("publish-post-2026-07-22-episode-036"), then even a model that erratically calls the tool three times produces exactly one publication. You've made the model's most characteristic failure — duplicated or repeated calls — structurally harmless. This is the same philosophical move as least privilege in the last lesson: don't make the model perfect; make its imperfections unable to cause damage.

5. Retry: Repeat Deliberately, Not Accidentally

With idempotency in place, retries become a tool rather than a hazard. But retrying well is its own small discipline, because the naive version — "if it fails, immediately try again, forever" — has destroyed real production systems.

The essentials:

Classify the failure first. Some errors mean "try again later" (timeout, 503, rate limit). Some mean "trying again is pointless" (validation failure, permission denied, resource doesn't exist). Retrying a 400 Bad Request forty times does not eventually make the request good; it makes your logs long and your incident report embarrassing. A retry policy without error classification is just a stutter.

Back off, with jitter. When a service is struggling, a thousand clients retrying in synchronized lockstep is a second attack arriving just as the patient sits up. Exponential backoff (wait 1s, then 2s, then 4s…) spreads the load over time; jitter (randomize the waits) prevents the retries from synchronizing into waves. AWS has published extensively on this after learning it the expensive way; the "retry storm" that turns a small outage into a large one is a recurring character in postmortems across the industry.

Bound the attempts. Every retry loop needs a budget — a maximum number of attempts or a deadline — after which the operation is declared failed and *escalated* rather than repeated. Which raises the operator's question: escalated to what? A dead-letter queue, an alert, a human. This connects directly to the previous unit's approval gates: the retry budget is where automated persistence ends and human judgment begins, and choosing that boundary is a business decision, not a technical one. An agent that retries a failing payout silently for six hours has not been persistent; it has been hiding a problem from you.

For agent workflows specifically, there is one more subtlety worth naming: **retry at the step level, not the workflow level.** If step four of a seven-step workflow fails, the correct move is to retry step four — not to re-run the whole workflow from step one, which would repeat three actions that already succeeded. This sounds obvious written down. It is not what happens by default when a model is simply re-prompted with "that didn't work, try again," which frequently causes it to replay the entire sequence. The structure that makes step-level retry possible is the workflow spec from two units ago: named steps, explicit state, defined failure handling per gate. The spec wasn't bureaucracy; it was the retry architecture, arriving early.

6. Receipts: Evidence, Not Memory

A **receipt** is a durable, structured record of an action that was actually performed: what was requested, what was done, when, by whom, with what result, and — crucially — with what external identifier.

The physical-world intuition is exact. When a shop hands you a receipt, it is not being polite. The receipt exists because both parties know that memory is unreliable and disputes are inevitable. It is the artifact that lets you later prove *this specific transaction happened*, independent of anyone's recollection.

Agent systems need receipts more than human ones do, for a blunt reason: **the model's context window is not a memory.** It is a working buffer that gets truncated, summarized, and discarded. An agent that "remembers" it published the post because the publication is mentioned in its context will forget the moment the session ends or the context rolls over. If the only record of an action lives in the transcript, the action is, operationally speaking, unrecorded.

A good receipt has a few properties:

- **It is written by the tool layer, not the model.** The receipt records what the system *did*, not what the model *believes it did*. These diverge more often than is comfortable.
- **It carries the external identifier.** Not "published the post" but "created post `pst_8k2m1`, at 09:14:02 UTC, HTTP 201, in feed `main`." The identifier is what makes the receipt *actionable* — it's the handle you use for verification, reconciliation, and undo.
- **It is durable and append-only.** Receipts go in a store that outlives the session and cannot be quietly edited. An audit log you can rewrite is a diary, not a ledger.
- **It records failures too.** "Attempted charge, received timeout, outcome unknown" is one of the most valuable receipts in the system, because it flags exactly the ambiguous state that needs resolution.

Receipts are also where this unit joins hands with the previous one. Tool Use II gave you approval gates — a human authorizes the action before it happens. Receipts are the after-image: proof of what was actually done under that authorization. Approval without receipts is signing blank checks; receipts without approval is surveillance of a system nobody governs. A well-run agent has both, and together they form the audit trail — which is not compliance theater but the thing that lets you *delegate more over time*. You extend an agent's autonomy at exactly the rate its receipts prove it deserves it. That is also, you may notice, how organizations extend trust to people.

7. Verification: Confirm the World Actually Changed

The final discipline closes the loop. **Verification** means: after acting, independently check that the world is now in the state the action was supposed to produce.

Note what this is *not*. It is not reading the tool's return value — that's just the response, and Section 2 established that responses can lie by omission, get lost, or report success for an action that half-happened. Verification is a *separate read*, through a separate path where possible, asking the world directly: is the post actually live at its URL? Does the database row actually contain the new value? Did the scheduled job's output actually land in the bucket?

The pattern has an old name in systems engineering — *read-after-write* — and an older one in ordinary competence: *check your work*. Deployment pipelines have practiced it for years as health checks and smoke tests: pushing the new version is the action; the deploy isn't "done" until the health check passes, and if it doesn't, the system rolls back. The action and the confirmation are separate steps with separate evidence.

For agent workflows, verification carries a particular weight because it is the **antidote to hallucinated success**. A model can generate the sentence "I have published the post" with perfect fluency whether or not any post was published — fluency and truth are uncorrelated at exactly this junction. A workflow that treats the model's report as the source of truth has no defense. A workflow that requires a verification read — fetch the feed, find the post ID from the receipt, confirm it's present — has converted "the model says so" into "the world says so." This is the same move, one lesson early, that evaluations make for model *outputs*: never grade the claim; grade the artifact.

Verification is also what completes "Done" from the workflow-spec unit. A step is not done when the tool returned. It is not done when the model says so. It is done when verification confirms the postcondition — and the receipt records

both the action and the confirmation. That is the full sentence, and every word in it is load-bearing.

Two practical notes. First, verification has cost — an extra read, extra latency — so calibrate it to stakes: verify every payment, spot-check low-stakes writes, and let reconciliation jobs (periodic sweeps comparing receipts against reality) catch what per-action verification skips. Second, verification can itself fail transiently; a failed verification read means "status unknown," not "action failed," and the response is to re-verify, not to re-act. The disciplines compose: you retry the verification, idempotently.

8. The Loop, Assembled

The five disciplines are not a menu. They are stations on one loop, each covering a specific failure the others don't:

Discipline	Question it answers	Failure it prevents	When it runs
Validation	Should this action run, as specified, right now?	Garbage or implausible requests reaching the world	Before the action
Idempotency	Is it safe if this runs twice?	Duplicates from retries, replays, or confused callers	Designed in, always
Retry	What do we do when it fails?	Transient failures becoming permanent ones	On failure, bounded
Receipt	What actually happened?	Actions with no evidence; agent amnesia	After every attempt
Verification	Did the world really change as intended?	Silent partial failure; hallucinated success	After apparent success

Read the loop as a sentence: *validate the request, execute idempotently, retry with backoff on transient failure, write a receipt for every attempt, verify the outcome against the world, and record the verification in the receipt.* Any tool you would let an agent use unsupervised should be able to answer for all five columns. Most tools you'll encounter in the wild can answer for none, which tells you exactly where the engineering work is when you wrap them for agent use.

9. Three Cases Worked Through

Abstractions earn their keep in the concrete. Here is the loop applied to the three running examples.

9.1 The feed publisher

An agent drafts a post and publishes it to a public feed — your content pipeline's last mile, and outward-facing, so mistakes are visible.

Validation: schema (title, body, feed exist), semantic (body length within bounds, no empty post, target feed is the intended one and not, say, the production feed when you meant staging), precondition (no post with this content ID already published).

Idempotency: the publish call carries a deterministic key derived from the content — `publish-ep036-paper`. The feed service records processed keys. If the agent calls publish twice, or a retry fires after a timeout that was secretly a success, the feed gets exactly one post. Without this, every timeout is a coin-flip between "nothing happened" and "your subscribers just got the same post twice," and your feed's reliability reputation is being set by network weather.

Retry: on timeout or 5xx, retry with backoff, up to three attempts within two minutes. On 4xx, stop and surface — the request is wrong and repetition won't fix it.

Receipt: `{key: publish-ep036-paper, post_id: pst_8k2m1, feed: main, status: 201, at: 09:14:02Z, attempt: 2}`. Note the receipt records that success came on attempt two — the first attempt's timeout receipt sits beside it, which is how you later notice the feed service is getting flaky *before* it fully falls over.

Verification: fetch the public feed; confirm `pst_8k2m1` is present and renders. Only then is the step done, and only then does anything downstream (announcements, cross-posts) trigger.

9.2 The scheduled job

A job runs nightly: gather the day's items, process them, write a summary. Schedulers are humble infrastructure with two famous vices: they fire twice (clock skew, restart during the window, overlapping runs when tonight's job starts before last night's finished), and they silently don't fire at all.

Validation: on start, check preconditions — has tonight's run already completed? Is last night's still running? A **lock or lease** ("I claim the 2026-07-22 run") prevents overlap; this is validation against *time* as state.

Idempotency: key the entire run by its logical date — `daily-summary-2026-07-22` — not by wall-clock time of execution. A double-fire becomes a no-op; a re-run after a crash resumes or replaces cleanly rather than producing `summary-final-v2-(1).md`.

Retry: the interesting choice here is *granularity*. Retry individual items, not the whole run — one poisoned item shouldn't force reprocessing of nine hundred good ones. Items that exhaust their budget go to a dead-letter list in the receipt rather than blocking the run.

Receipt: the run report — started, finished, items processed, items failed, items skipped, output location. This is what turns the scheduler's second vice into a detectable event: a monitoring check that asks "does a receipt exist for yesterday?" catches the silent no-fire. **The absence of a receipt is itself a signal** — and it's a signal you can only read if receipts are written reliably, which is why they're written by the job frame, not left to the job's good mood.

Verification: confirm the summary artifact exists at its expected location and is non-empty and parseable. "The job exited 0" and "the output is good" are different facts; exit codes are the tool's claim, the artifact is the world's answer.

9.3 The database write

The foundational case, because both examples above decompose into it. An agent updates a record — a customer's status, a ledger entry, a piece of GBrain state.

Validation: schema and types, plus preconditions read from the database itself: the record exists, the current value is what the agent believes it is. This last check — **compare-and-set**, "update status to `active` *only if* it is currently `pending`" — deserves special attention, because it defends against a failure mode unique to slow reasoning loops: the world changing *between the model's read and its write*. The agent read the record thirty seconds ago; a human edited it since; an unconditional write would silently stomp the human's change. Compare-and-set makes staleness a visible failure instead of a silent overwrite.

Idempotency: prefer absolute writes ("set balance_adjustment for invoice `inv_991` to –40") over relative ones ("subtract 40"). Where the operation is inherently relative — ledger appends — use an idempotency key on the entry so replays don't double-post. Accountants solved this centuries ago: you don't edit the ledger, you append

uniquely-identified entries, which is why double-entry bookkeeping is quietly one of history's great idempotent data structures.

Retry: safe now, because the write is either absolute or keyed. Bounded attempts; on exhaustion, escalate with the receipt trail attached.

Receipt: the write log — record ID, old value, new value, actor, timestamp, key. Old value matters: it is what makes the action *reversible*, and reversibility is what makes it safe to delegate in the first place.

Verification: read the row back; confirm the new value. For high-stakes tables, a periodic reconciliation sweep compares receipts against table state and flags drift — the systemic backstop for anything per-write verification missed.

10. Durable Execution: The Industrialized Version

Everything above can be hand-built, and for early systems it should be — you learn the failure shapes by meeting them. But it is worth knowing that the industry has consolidated these patterns into a named category: **durable execution**, with Temporal as its best-known exemplar (descended from Uber's internal Cadence system, which ran this pattern at scale for years before it was a product).

The core idea: you write your workflow as ordinary code — step one, step two, step three — and the durable execution engine records every step's inputs and results as an event history as it runs. If the process crashes, the machine dies, or the workflow needs to pause for a week waiting on a human approval, the engine replays the history to restore the workflow to exactly where it was, and continues. Completed steps are never re-executed — their recorded results are replayed instead. Steps that fail are retried according to declared policies. The workflow becomes, in effect, *immortal*: it runs to completion regardless of what fails underneath it.

Look at that description through this paper's vocabulary and you'll see it is the five disciplines, made infrastructure: the event history is **receipts**, elevated to the mechanism of execution itself; step boundaries with recorded results are **idempotency** at the workflow level; retry policies are declared per step, not improvised; and the replay model enforces the step-level retry granularity that Section 5 argued for. The intellectual lesson is that these are not five tips — they are one coherent theory of reliable action, coherent enough that an entire product category is built on it.

For the operator's map: durable execution engines answer "how do multi-step workflows survive failure," and they are increasingly the substrate underneath serious agent systems, because an agent workflow — long-running, gated on human approvals, full of fallible external calls — is precisely the workload they were designed for. You do not need Temporal to apply this paper. You need this paper to understand why Temporal exists.

11. Where This Sits in an AI-Native Company

Briefly — the mechanism is the lesson; the framing is context.

An orchestration layer like FRIDAY is, at bottom, a system that turns intentions into actions through tools. Its trustworthiness is exactly the trustworthiness of its action loop. Memory and knowledge (GBrain) tell it what's true; permissions tell it what's allowed; but validation, idempotency, retry, receipts, and verification are what make its *actions* dependable — and receipts specifically are what let its actions become *institutional memory* rather than session vapor. When a future session asks "did we already publish this?", the answer must come from the receipt store, not from anyone's — human or model's — recollection.

There is also a hiring-manager way to see this unit. You extend an employee's autonomy based on evidence of reliable execution, and you build that evidence from their track record. Receipts and verification are the agent's track record, produced automatically. The organizations that get durable value from agents will be the ones that built the evidence loop, because the evidence loop is what makes expanding delegation a calculated decision instead of a leap of faith. The ones that skipped it will alternate between over-trusting and banning, which is not a strategy; it is a mood.

12. Limitations and Honest Edges

The loop is powerful, not magic. Its edges are worth knowing.

Verification confirms *that*, not *whether it was wise*. The full loop can flawlessly validate, execute, and verify the publication of a post that should never have been published. Reliability disciplines guarantee the action happened as specified; they say nothing about whether the specification was good. Judgment about *what* to do remains the province of evaluations, approval gates, and humans. Do not let a beautiful receipt trail lull you into skipping the question "was this the right action?"

Some actions cannot be made idempotent, only compensated. An email, once sent, is sent. A tweet, once seen, is seen. For these, the pattern degrades to compensation — send a correction, issue a refund, publish a retraction — which is visible and imperfect. The practical consequence: reserve genuinely irreversible actions for the highest gates, because your reliability toolkit is weakest exactly there.

Receipts and verification cost real money and latency. Every receipt is a write; every verification is a read; retries multiply load. Calibrate to stakes rather than gold-plating everything — a system that verifies every trivial write is paying insurance premiums on paperclips.

The reliability machinery is itself software, and can fail. The receipt store can go down; the idempotency-key table can fill; the verification read can hit a stale replica and report false failure. This does not undermine the approach — it means the loop's components need the same operational care as the actions they protect, and it is why buying the battle-tested version (Section 10) eventually beats maintaining a bespoke one.

None of these edges is a reason to skip the loop. They are reasons to apply it with a sense of proportion — which is, after all, the operator's actual job.

13. Vocabulary

Validation — Checking a requested action before execution: schema (right shape), semantics (plausible values), preconditions (coherent with current world state).

Idempotency — The property that performing an operation more than once has the same effect as performing it once. The precondition for safe retries.

Idempotency key — A caller-supplied unique token attached to a request, letting the server detect and deduplicate repeats by returning the original result instead of re-executing.

Retry — Deliberately re-attempting a failed operation, governed by error classification, exponential backoff with jitter, and a bounded budget with escalation.

Exponential backoff / jitter — Progressively longer, randomized waits between retries, preventing retrying clients from synchronizing into destructive waves.

Receipt — A durable, structured, append-only record of an attempted action: request, outcome, timestamp, actor, and external identifier. Written by the tool layer, not the model.

Verification — An independent post-action read confirming the world reached the intended state; the defense against lost responses, partial failure, and hallucinated success.

Compare-and-set — A conditional write ("update only if the current value is X") that turns stale-read conflicts into visible failures rather than silent overwrites.

Dead-letter queue — Where operations go after exhausting their retry budget, awaiting inspection instead of blocking the workflow or failing silently.

Reconciliation — A periodic sweep comparing receipts against actual world state, catching drift that per-action verification missed.

Durable execution — An execution model (Temporal, Cadence) in which a workflow's step-by-step history is persisted as it runs, so the workflow survives crashes by replaying completed steps' recorded results and resuming.

Compensation — The fallback for irreversible actions: a visible corrective action (refund, retraction) rather than an undo.

14. Reading Guide

Temporal — durable execution (temporal.io). Read the homepage and the "How Temporal works" material with one question in mind: *which of this paper's five disciplines is each feature implementing?* You should be able to map event history → receipts, workflow replay → workflow-level idempotency, activity retry policies → declared retries, and activity boundaries → the step-level retry granularity from Section 5. Skim a code example even if you don't write the language — the striking thing is how *ordinary* the workflow code looks, because the reliability machinery lives in the engine rather than the business logic. That separation — reliability as substrate, not as scattered if-statements — is the architectural taste worth absorbing. Twenty to thirty minutes; do not descend into deployment documentation, which is for a different day and a different mood.

Optional, if appetite remains: search for Stripe's engineering post on idempotency keys ("Designing robust and predictable APIs with idempotency"). It is short, concrete, and shows the pattern of Section 4 as practiced by a company for which a duplicate action is a duplicate charge.

15. Walking Questions

1. Take one action you'd want an agent to perform in a real workflow — publishing, messaging, updating a record. Walk it through all five columns of the table in Section 8. Which column can you not yet answer? That gap is the engineering work.

2. Explain, out loud, why a timeout is more dangerous than an error. If your explanation includes the phrase "you can't tell whether it happened," you have the core of the episode.

3. Where in FRIDAY or GBrain does an action currently happen with *no receipt* — no durable record outside the conversation? What would you fail to prove about that action a month from now?

4. Pick one bounded workflow — the feed publisher is a fine candidate — where you could add idempotency keys and a receipt log this month as a safe, small test. What would the receipt schema contain? What single verification read would close the loop?

5. Which action in your current stack is genuinely irreversible? Given Section 12, does it sit behind a gate proportional to that irreversibility?

16. One-Sentence Takeaway

An agent's action is trustworthy not when the model says it succeeded, but when the request was validated, the operation was safe to repeat, failures were retried deliberately, a durable receipt recorded what happened, and an independent check confirmed the world actually changed.